# Further Kokkos

Evan Drake Suggs
ASCEND Research group - TN Tech
March 9th, 2026

# Clarifying subviews and make_pair

- To clarify the logic for subviews pairs, a `subview(A, make_pair(0, 5))` creates a 1D subview from indices 0 to indices 4, while `subview(A, make_pair(1,5))` makes one from indices 1 to indices 4.
- This is because `make_pair` creates a range, for example `make_pair(1, 5) = subview.extent(0) = 5 - 1 = 4`.
- Also, note that you will need to use cuda::pair/cuda::make_pair when working on Kokkos+Cuda problems, Kokkos::pair should work also.
- MDSpan provides similar functionality, pair(1, 5) will to give indices 1 to 4.

# Clarifying subviews and make_pair

- As shown here, the original View above, with the subview with extents.
- The following shows rows 0 to 4 and columns 1 to 3.

```
 1  2  3  4  5  6  7  8
 2  3  4  5  6  7  8  9
 3  4  5  6  7  8  9  10
 4  5  6  7  8  9  10  11
 5  6  7  8  9  10  11  12
 6  7  8  9  10  11  12  13
 7  8  9  10  11  12  13  14
 8  9  10  11  12  13  14  15
Extent 0: 5 make_pair(0,5);
Extent 1: 3 make_pair(1, 4);
 2  3  4
 3  4  5
 4  5  6
 5  6  7
 6  7  8
Goodbye World
```

# Review

- Kokkos has two primary features, data management via the View and parallel dispatch via the parallel_for, reduce, and scan operations
- Kokkos extends the C++ language via compile time features, along with supporting a large variety of backends, such as Cuda and OpenMPI
- This lecture primarily covers the Kokkos template system and how to use it properly

# View Templates

- A major part of Kokkos's efficiency lies in its advanced metaprogramming techniques
- Metaprogramming is code that works on code
- For example, templates can be used to manipulate C++ code at compile time
- We have covered the most common use-case where a View is templated with a datatype followed by a description of dimensions (i.e. `Kokkos::View<int*>`)
- There are many other templates in Kokkos, hidden behind default parameters

# View Layouts

- A simple example of default template parameter is what variety of layout a View has
- Primary Layouts Types: the C-style row-major LayoutLeft, the Fortran-style column-major LayoutRight or LayoutStride
- `Kokkos::View<int[5],Kokkos::LayoutLeft> != Kokkos::View<int[5],Kokkos::LayoutStride>`
- Mixing them can result in memory leaks or incorrect indexing

# 2d Memory Layouts

Row-Major (Left)

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |

Column-Major (Right)

| | | |
|---|---|---|
| 1 | 4 | 7 |
| 2 | 5 | 8 |
| 3 | 6 | 9 |

LayoutStride

| | | |
|---|---|---|
| 1 | | 2 |
| 3 | | 4 |
| 5 | | 6 |

# View templating and functions

- Templating must be used when a View is a template parameter, or the function will only work for the defined View (for example, View<int*>)
- This can be worked around via typename (for example, `template<typename View_t>`)
- You might have to instantiate the templates for larger projects
- You can specialize with techniques like “requires” or “enable_if” for different template parameters

# View templating and functions

```
// Wrong way
int foo(Kokkos::View<double*> A){}
// only works if A is 1D double View
template<typename View_t>
int foo(View_t A)
// may require instantiation
template int foo(Kokkos::View<int*> A);
```

# View functions and specialization

```
// requires means that for it to be compiled,
// one View must have char as its datatype
template<typename View_t> requires std::is_same_v
          <typename View_t::value_type, char>
void foo(View_t& buf){...}

// default
template<typename View_t>
void foo(View_t& buf){...}
```

# View Spaces

- The memory and execution space can be specified
- The execution and memory space also affect other defaults, namely the View's layout

```
Kokkos::View<int*> a(“a”, 100000);
Kokkos::View<int*, Kokkos::HostSpace> b ("b", 100000);
Kokkos::View<int*, Kokkos::CudaSpace> c ("c", 100000);
Kokkos::View<int*,
Kokkos::Device<Kokkos::Cuda,Kokkos::CudaUVMSpace> > a
                              ("a", 100000);
```

# Kokkos Timer

- Kokkos has a built-in timer that outputs the time in seconds
- Timer can be reset for use later

```
Kokkos::Timer timer;
// do something
printf("seconds: %f", timer.seconds);
timer.reset()
//something else
printf("seconds: %f", timer.seconds);
```

# View loop tricks

- All Views map to 1D in memory, just like any other array
- Be careful about knowing your layout so your logic works, especially if an index is dependent on another
- The first example launches n processes that run through n iterations, while the second launches n*n processes that do only 1 iteration.
- How does the finished View differ?
- What's stopping you from doing nested parallel fors? Nothing, but it might just complicate things

# View loop fusion

```
Kokkos::View<int**> check(“check”, n, n);

Kokkos::parallel_for(“iterator”, n, KOKKOS_LAMBDA(
    Const int& i){
    for(int j=0; j<n; j++){check(i,j) = i;
}   });
Kokkos::parallel_for (“ unrolled iterator”, n*n,
KOKKOS_LAMBDA(const int& i){
    check(i) = i;
});
```

# Hierarchical Parallelism and advanced techniques

- Kokkos allows the creation of kernels that use hierarchical parallelism, that is parallelism where tasks are designated separately on different levels of hardware (different sockets, cores, etc.)
- This is done using Team policies to create work teams and more advanced policies, such as ranges
- Kokkos::single(Policy, lambda) allows you to atomically do lambdas also

# More about Lambdas and reductions

```
typedef TeamPolicy<ExecutionSpace>::member_type member_type;
// Create an instance of the policy
TeamPolicy<ExecutionSpace> policy (league_size, Kokkos::AUTO() );
// Launch a kernel
parallel_for (policy, KOKKOS_LAMBDA (member_type team_member) {
    // Calculate a global thread id
    int k = team_member.league_rank () * team_member.team_size () +
            team_member.team_rank ();
    // Calculate the sum of the global thread ids of this team
    int team_sum = k;
    team_member.team_reduce(Sum<int, typename
ExecutionSpace::memory_space>(k));
    // Atomically add the computed sum to a global value
    Kokkos::single (PerTeam (team_member), [=] () {
      atomic_add(&global_value(), team_sum);
    });
  });
```

# KokkosComm

- KokkosComm is an experimental library that integrates Kokkos and MPI
- This allows integration of off-node communication and on-node computation while maintaining performance portability
- This is in contrast to the traditional way of doing MPI+Kokkos, sending the underlying data buffer using the .data method.
- KokkosComm also features a variety of other features, like packaging non-contiguous Views, and additional backends such as NVIDIA Collective Communications Library

# KokkosComm example

```
// from
https://github.com/kokkos/kokkos-comm/blob/develop/perf_tests/mpi/test_sen
drecv.cpp
MPI_Init(&argc, &argv);
Kokkos::initialize( argc, argv );
{
     int rank, size;
     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
      MPI_Comm_size(MPI_COMM_WORLD, &size);
     Kokkos::View<int***> check("check", n, n, n);
     // traditional way
     MPI_Send(&check.data(), n*n*n, MPI_INT, 1, 0, MPI_COMM_WORLD);
     // kokkos comm
     KokkosComm::mpi::send(space, check, 1, 0, comm, Mode{});
}
Kokkos::finalize();
MPI_Finalize();
```

# Getting Started

- First is to compile Kokkos if not included on your system
- Can also be done with Spack

```
mkdir build
cd build
cmake ${srcdir}
 -DCMAKE_CXX_COMPILER=g++
 -DCMAKE_INSTALL_PREFIX=${my_install_folder}
```

# Getting Started on Cluster

- TN Tech's cluster has a spack module for Kokkos already.

```
spack load kokkos@4.7~cuda gcc@14 openmpi cmake

cd /work/classes/csc4760-001-2026s/<username>

hpcshell --account=csc4760-001-2026s --cpus-per-task=2

Or salloc --account=csc4760-001-2026s --cpus-per-task=2
```

# Additional Kokkos Tools

- Kokkos kernels: focuses on math kernels and Basic Linear Algebra Subprograms (BLAS)
- Kokkos remote spaces: adds global and distributed shared memory support
- PyKokkos: Python bindings for Kokkos

# Kokkos in Action: Trilinos

- Trilinos: a collection of scientific libraries, including linear algebra solvers, calculus functions, and sparse matrix tools
- Uses Kokkos for extending BLAS and sparse matrices
- The Kokkos View forms the backbone of many Trilinos tools but is not accessible to the user

# Kokkos in Action: Cabana

- Cabana: Library that uses Kokkos and MPI to build particle simulations
- Extends Views to provide particle grids and meshes, along with algorithms to support them
- Bundles initialize and finalize with its ScopeGuard function

# Further Reading on Kokkos and Metaprogramming

For further information on Kokkos, the Kokkos programming guide and tutorials

https://kokkos.github.io/kokkos-core-wiki/index.html

https://github.com/kokkos/kokkos-tutorials

Metaprogramming:

https://en.cppreference.com/w/cpp/language/templates

(Optional) C++ Templates The Complete Guide